CORHUILA
CORPORACIÓN UNIVERSITARIA DEL HUILA
Vigilada Mineducación

INSTITUCIÓN DE EDUCACIÓN SUPERIOR SUJETA A INSPECCIÓN
Y VIGILANCIA POR EL MINISTERIO DE EDUCACIÓN NACIONAL – SNIES 2828

PlanificaIA: agente inteligente para la gestión de tareas académicas

Estudiantes

Dayana Sofía Méndez Perafán

Sneyder Díaz

Docente

José Miguel Llanos Mosquera

Corporación Universitaria del Huila - CORHUILA

Facultad de Ingeniería

Ingeniería de Sistemas

Electiva Aplicaciones con Inteligencia Artificial

Neiva, Huila, 2026

Resumen

PlanificaIA es un agente académico desarrollado en Python 3 para registrar tareas, conversar con el estudiante y apoyar la elección de la siguiente actividad. El sistema combina reglas de priorización con un modelo de lenguaje accesible mediante Groq. La aplicación permite crear, modificar y completar tareas; consultar pendientes; visualizar un calendario mensual; y exportar un informe en PDF de las tareas por hacer. El navegador admite dictado y lectura de respuestas en voz alta cuando ofrece esas funciones. La lógica del servidor se implementó en Python y la interfaz web se publicó en Render. Se verificó que el sitio público carga, informa que Groq está configurado y responde a una consulta de prueba. En consola se registraron dos tareas de ejemplo y el agente recomendó la de entrega más cercana. Las tareas del tablero se almacenan en el navegador de cada usuario, por lo que no se sincronizan entre equipos. La clave API permanece en la configuración privada del servidor.

Palabras clave: agente, tareas, priorización, educación, Groq.

Abstract

PlanificaIA is an academic task agent built with Python 3. It records assignments, chats with students, and helps them decide what to work on next. The system combines transparent priority rules with a language model accessed through Groq. Students can create, edit, and complete tasks; review pending work; inspect a monthly calendar; and export a PDF report of unfinished tasks. Compatible browsers also support speech input and spoken responses. The Python server and web interface were deployed on Render. The public website was observed loading successfully, reporting an active Groq configuration, and responding to a sample question. A command line check added two example tasks and recommended the one with the earlier deadline. Task records are stored separately in each user's browser and are not synchronized across devices. The API key remains in the server's private configuration.

Keywords: agent, tasks, prioritization, education, Groq.

Introducción

Un estudiante puede tener entregas, lecturas y exposiciones con fechas y niveles de urgencia distintos. Cuando la información está dispersa, se dificulta saber por dónde comenzar. PlanificaIA reúne esos pendientes en una sola interfaz y ofrece una recomendación explicada a partir de los datos ingresados.

El proyecto se desarrolló como un agente de software: recibe información sobre tareas, mantiene un estado de pendientes y completadas, y ejecuta acciones como registrar, consultar, modificar y priorizar. El componente conversacional interpreta solicitudes en lenguaje natural y puede usar herramientas del servidor para actualizar la lista (Groq, s. f.).

Justificación

La organización académica es un problema concreto que permite mostrar percepciones, decisiones y acciones del agente. Una regla transparente mantiene útil el sistema si el servicio generativo no está disponible. La versión web facilita la demostración durante la sustentación y el calendario hace visibles las fechas críticas.

Objetivos

Objetivo general

Desarrollar y publicar un agente en Python 3 que gestione tareas académicas y oriente al estudiante sobre cuál atender primero mediante una interfaz web y una versión de consola.

Objetivos específicos

Definir los datos y reglas de gestión de tareas; implementar acciones para crear, consultar, modificar y completar registros; integrar un chat con Groq para interpretar solicitudes; presentar calendario, historial y exportación de pendientes en PDF; y verificar los flujos principales en consola y en el sitio publicado.

Alcance

Cada tarea contiene título, fecha límite opcional, prioridad, categoría, notas, tiempo estimado y estado. El tablero permite buscar y filtrar. Los datos del navegador persisten localmente entre sesiones mediante *localStorage* (MDN Web Docs, s. f.). El proyecto no incluye cuentas ni sincronización entre dispositivos.

Desarrollo

Diseño del agente

El usuario expresa una necesidad por texto o voz. La interfaz envía el mensaje y las tareas pendientes al servidor Python. El servidor consulta Groq cuando existe una clave configurada; el agente puede responder, solicitar datos faltantes o proponer acciones sobre las tareas. La lista actualizada vuelve al navegador y se conserva allí.

Las acciones disponibles son agregar, modificar, completar, listar y recomendar la siguiente tarea. El agente debe validar el título, la fecha y la prioridad antes de aplicar un cambio. La respuesta muestra qué tarea se creó o modificó, lo que permite al estudiante reconocer el resultado.

Reglas de priorización

La recomendación local considera primero tareas vencidas, después la fecha límite más cercana y finalmente la prioridad como criterio de desempate. Las tareas sin fecha quedan detrás de las fechadas. Este orden es verificable y evita presentar una sugerencia sin explicación.

Construcción del agente

Pasos realizados

Primero se eligió el tema y se acotó el alcance a tareas académicas individuales. Después se modelaron los campos, los estados y la validación. Se programó *agente_tareas.py* con las operaciones y el modo de consola; *web_app.py* expuso la página y los puntos de acceso HTTP. La interfaz *dist/index.html* incorporó formulario, chat, dictado, lectura de voz, historial, calendario, estadísticas y exportación PDF.

A continuación se configuró el repositorio GitHub del grupo y un Blueprint de Render. El archivo *render.yaml* define un servicio web Python, el comando de instalación de dependencias, el inicio del servidor y una ruta de salud. *GROQ_API_KEY* se declara sin valor en el código para introducirla de manera privada durante el despliegue (Render, s. f.).

Persistencia y seguridad

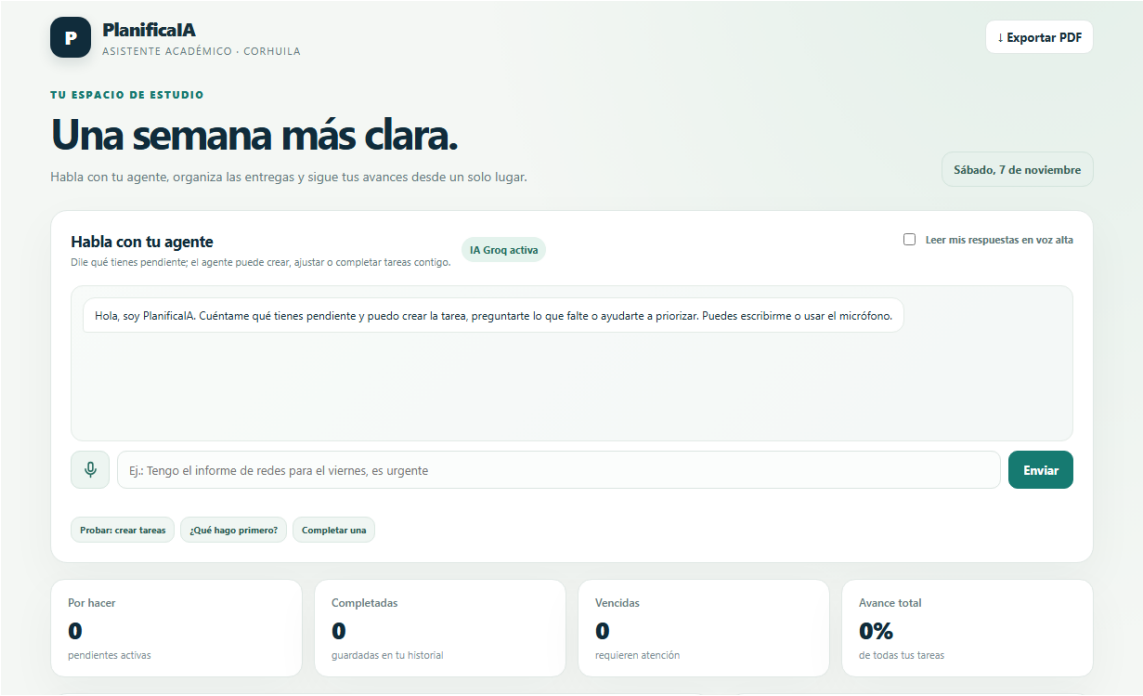
Las tareas web se guardan en *localStorage*; las operaciones de consola utilizan almacenamiento JSON local. La clave API no se almacena en el HTML ni se subió al repositorio. El chat envía al servidor el mensaje y las tareas pendientes necesarias para procesar cada solicitud.

Interfaz web publicada

La página pública presenta primero el chat con el agente, seguido de indicadores de progreso, un formulario para tareas y el calendario. También ofrece exportar solo los pendientes en formato PDF desde la ventana de impresión del navegador.

Figura 1

Interfaz pública de PlanificaIA con chat e indicadores de progreso.



Nota. Captura de https://planificaia.onrender.com/ realizada el 7 de noviembre de 2026. El indicador muestra que la clave de Groq está configurada en el servidor.

Pruebas funcionales

Consola

Se ejecutó una copia aislada del programa para no alterar los datos del proyecto. Se agregaron dos actividades ficticias, se listaron y se solicitó una recomendación. El orden esperado colocaba primero la exposición del 9 de noviembre frente a la lectura del 12 de noviembre.

Registro 1. Fragmento de la salida obtenida en consola.

```
Tú > agregar Preparar exposición | 2026-11-09 | alta
Agregada [182a4ee9]: Preparar exposición
Tú > agregar Leer capítulo | 2026-11-12 | media
Agregada [01d389fd]: Leer capítulo
Tú > recomendar
Agente (Reglas locales) > Te recomiendo empezar por Preparar exposición.
```

Chat publicado

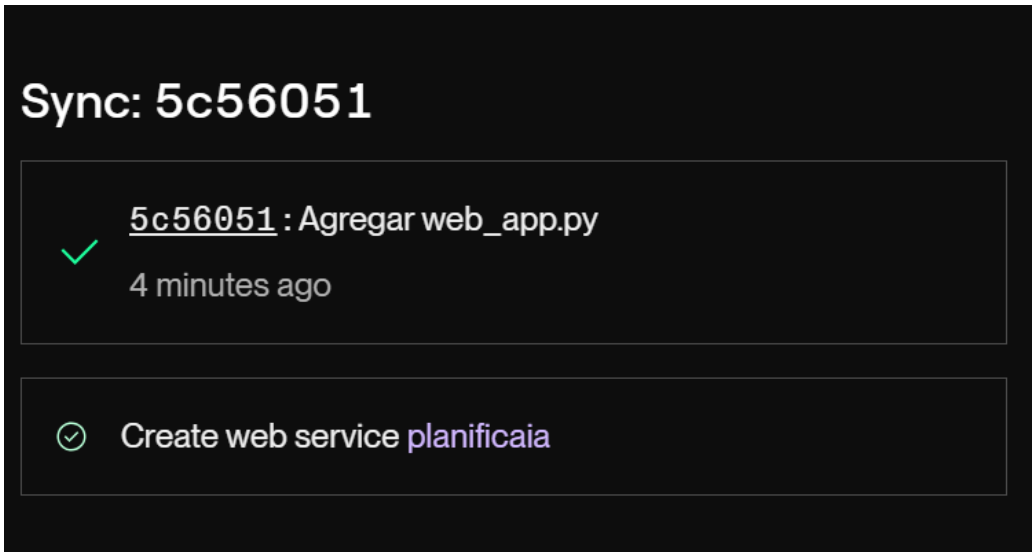
En el sitio desplegado se envió la pregunta «Hola, ¿qué puedes hacer con mis tareas?». Groq respondió que puede crear, modificar, completar y listar tareas, además de recomendar una según urgencia y prioridad. Esta prueba verificó una respuesta efectiva del chat, más allá del indicador de configuración.

Resultados obtenidos

El repositorio del grupo quedó publicado en GitHub y Render creó el servicio web *planificaia*. El despliegue figura en estado *Live* y la dirección https://planificaia.onrender.com/ abrió la interfaz sin iniciar sesión. Se observó el chat activo, el calendario y el botón de exportación PDF.

Figura 2

Sincronización del Blueprint y creación del servicio web.



Nota. Captura del panel de Render proporcionada por el grupo el 7 de noviembre de 2026. El estado posterior *Live* se comprobó en el servicio.

Análisis de los resultados

La versión de consola mostró que las reglas locales permiten priorizar dos tareas con fechas distintas. La página pública confirmó que el servicio está disponible y que el chat responde mediante Groq. Estos resultados respaldan el objetivo de ofrecer una ayuda práctica para registrar pendientes y decidir el siguiente paso.

La prueba de voz depende del permiso del micrófono y de las funciones del navegador. El dictado y la síntesis de voz están implementados en el cliente, pero no se documenta una prueba de micrófono en este informe. La exportación PDF usa el diálogo de impresión del navegador; el usuario debe elegir «Guardar como PDF».

El almacenamiento local protege la sencillez del prototipo, aunque impide que dos personas vean la misma lista en dispositivos diferentes. También significa que la demostración del profesor comenzará vacía en su navegador. En el plan gratuito de Render el servicio puede tardar en reactivarse tras un período sin uso.

Conclusiones

Se desarrolló un agente académico con una versión de consola y una interfaz web pública. El sistema registra y completa tareas, conserva el historial en el navegador, muestra un calendario y genera un informe PDF de pendientes. El chat publicado respondió una consulta de prueba mediante Groq.

Las reglas de priorización explican la recomendación con fechas y prioridades comprensibles. La combinación de interfaz visual y conversación facilita presentar el comportamiento del agente durante la sustentación. Su alcance permanece individual porque no existe sincronización entre usuarios.

Recomendaciones

Para la sustentación se recomienda crear tareas ficticias en el navegador que se mostrará al docente, enseñar una entrega vencida en el calendario, completar una tarea y guardar el PDF de pendientes. Conviene abrir el sitio unos minutos antes por la posible espera inicial del plan gratuito. Como trabajo posterior, se propone una base de datos con cuentas y sincronización de tareas.

Referencias

Corporación Universitaria del Huila. (2026). *Parcial práctico tercer corte: Aplicaciones con Inteligencia Artificial* [Guía de evaluación].

Groq. (s. f.). *Text generation*. GroqDocs. https://console.groq.com/docs/text-chat

MDN Web Docs. (s. f.). *Window: localStorage property*. https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage

Méndez Perafán, D. S., & Díaz, S. (2026). *PlanificaIA* [Código fuente]. GitHub. https://github.com/DayanaMendess/PlanificaIA

Python Software Foundation. (s. f.). *http.server - HTTP servers*. Python documentation. https://docs.python.org/3/library/http.server.html

Render. (s. f.). *Blueprint YAML reference*. https://render.com/docs/blueprint-spec